DATA STRUCTURES – QUESTIONS AND ANSWERS

Question 1

Given: ((A + B) / C) ^ ((D - E) * F). Convert the expression to postfix manually and explain the nested structure handling.

Answer

The postfix form of the given infix expression is:

```
A B + C / D E - F * ^
```

Manual conversion

1. Convert the innermost addition (A + B) into postfix: **AB+**.
2. Divide that result by C: **AB+C/**.
3. Convert the subtraction (D - E) into postfix: **DE-**.
4. Multiply that result by F: **DE-F***.
5. Apply the exponent operator to the two completed sides: **AB+C/DE-F*^**.

How nested structure is handled

Parentheses define the order of evaluation. Each parenthesized subexpression is converted before the operator outside it is processed. The left side, ((A + B) / C), becomes **AB+C/**. The right side, ((D - E) * F), becomes **DE-F***. Finally, the exponent operator (^) is placed after both operands. In a stack-based conversion, operators are temporarily pushed onto a stack; parentheses control when operators are popped, ensuring that inner expressions are completed before outer operations.

Final answer: `AB+C/DE-F*^`

Question 2

How would you integrate binary search in a product inventory system with real-time price lookup? Explain the design.

Answer

Binary search can be used to quickly locate a product in an inventory that is kept sorted by a unique key, such as Product ID or SKU. Once the product record is found, the system retrieves its latest price from the price store or pricing service.

System design

1. **Store product data:** Maintain product records with Product ID, name, stock quantity and a reference to the current price.
2. **Keep the search index sorted:** Store Product IDs in ascending order, with each ID linked to its product record. The index must remain sorted for binary search to work.
3. **Search:** Compare the requested Product ID with the middle ID. If it matches, the product is found. If the requested ID is smaller, search the left half; otherwise search the right half. Repeat until found or the range becomes empty.
4. **Get the current price:** After finding the product, request its latest price from the pricing database/service. A short-lived cache may improve speed, but it must be updated or invalidated whenever the price changes.
5. **Display the result:** Return the product details, current price, and availability. If the product ID is not

found, display a suitable “Product not found” message.

Example

```
Sorted Product IDs: 101, 115, 120, 135, 150, 172, 190
Search for Product ID: 150
Middle ID = 135; 150 is greater, so search the right half.
Next middle ID = 172; 150 is smaller, so search the left part.
Next ID = 150; product found. Retrieve its latest price.
```

Advantages and important points

• Binary search takes **O(log n)** comparisons, which is efficient for large sorted inventories.
• Looking up the latest price is a separate step; binary search does not itself make a price real-time.
• Sorting by a stable key such as Product ID is usually better than sorting by price, because frequent price changes would otherwise require reordering the index.
• When products are added or removed, update the sorted index. For very large or frequently changing inventories, a database index or balanced search structure may be more suitable.

Conclusion: Use binary search on a sorted Product ID index to locate the product quickly, then fetch the latest price from the pricing service/database to provide an up-to-date result.